

BLP File Format Specification

Format: Blizzard BLP Texture Format (Warcraft III / World of Warcraft) **Byte Order:** Little-endian **File Magic:** **BLP0**, **BLP1**, or **BLP2** **Supported Versions:** BLP0 (WC3 Beta), BLP1 (Warcraft III), BLP2 (World of Warcraft)

This document describes the BLP (Blizzard Picture) texture format used across multiple Blizzard Entertainment titles. BLP is an image container format that stores a single image with optional mipmaps, supporting several encoding modes including palettized color, JPEG compression, DXT/S3TC block compression, and uncompressed BGRA.

Authorship attribution: Fernando A. Sahmkow

Revision History

Date	Version	Description
2026-03-08	1.0	Initial comprehensive specification compiled from wowdev.wiki, HiveWorkshop (Dr Super Good), Wowpedia, and the image-blp Rust crate documentation

Table of Contents

1. [Overview](#)
2. [Conventions](#)
3. [File Structure](#)
4. [BLP1 Header](#)
5. [BLP2 Header](#)
6. [Mipmap Location Table](#)
7. [Content Headers](#)
8. [Mipmap Data](#)
9. [Encoding Formats](#)
10. [Version Differences](#)
11. [Implementation Notes](#)

Appendices

- [A — Mipmap Scaling Logic](#)
 - [B — DXT Block Compression Details](#)
 - [Credits & References](#)
-

1. Overview

BLP (Blizzard Picture) is a **binary texture format** created by Blizzard Entertainment. It stores a single image with an optional full set of mipmap levels. The format supports images up to 65,535 pixels in either dimension (though practical limits are often lower depending on game version and GPU capabilities). All multi-byte values are **little-endian**.

BLP files support several pixel encoding modes:

- **JPEG** — Non-standard JPEG compression using BGRA color components instead of Y’CbCr.
- **Palettized (RAW1)** — Indexed color using a 256-entry palette with separate alpha channel.
- **DXT/S3TC** — Block-compressed textures (DXT1, DXT3, DXT5) for GPU-native decompression.
- **Uncompressed BGRA (RAW3)** — Raw 32-bit pixels stored directly.

Known Versions

Magic	Version	Game	Notes
BLP0	0	Warcraft III: Reign of Chaos (beta)	Mipmaps stored in external files (.b00–.b15)
BLP1	1	Warcraft III: Reign of Chaos / The Frozen Throne	Internal mipmaps. Supports JPEG and palettized encodings
BLP2	2	World of Warcraft (all versions)	Internal mipmaps. Adds DXT compression and uncompressed BGRA

Colour space note: BLP files technically contain linear RGB colour space images. However, Blizzard treated image components as sRGB when encoding Warcraft III assets. For practical compatibility, it is recommended **not** to perform colour space correction when processing BLP files. If a colour space is required for output, sRGB should be assumed.

2. Conventions

Data Types

Notation	Size	Description
u8	1	Unsigned 8-bit integer
u16	2	Unsigned 16-bit integer
u32	4	Unsigned 32-bit integer
char[N]	N	Fixed-length ASCII string
u8[N]	N	Array of N unsigned bytes
u32[N]	4×N	Array of N u32 values

Notation

- (X) means field or structure X is **optional** or **conditional**.
- [N] after a type means a fixed-length array of N elements.
- // condition annotates when a field is present or how to interpret it.
- All offsets are from the start of the file unless otherwise noted.
- Hexadecimal values are prefixed with 0x.

Magic Number Encoding

The magic field consists of 4 ASCII characters stored sequentially. In code:

```
constexpr u32 decodeMagic(char a, char b, char c, char d) {
    return static_cast<u32>(a) | (static_cast<u32>(b) << 8) |
           (static_cast<u32>(c) << 16) | (static_cast<u32>(d) << 24);
}

constexpr u32 BLP0_MAGIC = decodeMagic('B', 'L', 'P', '0'); //
0x30504C42
constexpr u32 BLP1_MAGIC = decodeMagic('B', 'L', 'P', '1'); //
0x31504C42
constexpr u32 BLP2_MAGIC = decodeMagic('B', 'L', 'P', '2'); //
0x32504C42
```

3. File Structure

The BLP file is structured as a header followed by content-specific metadata and mipmap image data. The exact layout varies by version.

BLP1 File Layout

```
BLP1_File {
    BLP1Header          header          // 28 bytes (0x00-0x1B)
    MipmapLocator        mipmapLocator    // 128 bytes (0x1C-0x9B)
    ContentHeader        contentHeader    // Varies by content type
    MipmapData[0..15]    mipmapChunks    // Image data at locations
    specified by mipmapLocator
}
```

BLP2 File Layout

```
BLP2_File {
    BLP2Header          header          // 1172 bytes (0x00-0x493),
    includes mipmap locator and palette
    MipmapData[0..15]    mipmapChunks    // Image data at locations
    specified by header.mipOffsets
}
```

The BLP2 header is a single monolithic structure. See [Section 5](#) for the full field layout.

BLP0 File Layout

```
BLP0_File {
    BLP0Header          header          // 28 bytes (0x00-0x1B)
    ContentHeader        contentHeader    // Varies by content type
    // Mipmap data stored in external files: <basename>.b00 through
    <basename>.b15
}
```

4. BLP1 Header

Applies to: BLP0 and BLP1 files **Size:** 28 bytes

```
BLP1Header {
    char[4]    magic           // +0x00  "BLP0" or "BLP1"
    u32        content         // +0x04  Content type
    u32        alphaBitDepth   // +0x08  Alpha bits per pixel: 0, 1,
4, or 8
    u32        width           // +0x0C  Image width in pixels
    u32        height          // +0x10  Image height in pixels
    u32        extra           // +0x14  Typically 4 or 5; purpose
unclear
    u32        hasMipmaps      // +0x18  0 = no mipmaps, non-zero =
mipmaps present
}
```

4.1 Content Type

Value	Name	Description
0	CONTENT_JPEG	JPEG-encoded content using non-standard BGRA colour components
1	CONTENT_DIRECT	Directly encoded content (palettized / indexed colour)

If the `content` field contains an invalid value, `CONTENT_JPEG` (0) must be assumed. It is recommended to generate a warning in this case.

4.2 Alpha Bit Depth

Value	Description
0	No alpha channel. Image is fully opaque
1	1-bit alpha. 8 pixels packed per byte, LSB first
4	4-bit alpha. 2 pixels packed per byte, low nibble first
8	8-bit alpha. One byte per pixel, full 0–255 range

For `CONTENT_JPEG`, only 0 and 8 are valid (due to JPEG component mechanics). Even when `alphaBitDepth` is 0, the JPEG alpha component must physically exist in the encoded data.

Invalid values for the given content type must evaluate as 0 (fully opaque), and a warning should be generated.

4.3 Width and Height

Pixel dimensions of the full-scale image (mipmap level 0). Although often a power of two, this is not strictly required. The `u32` type allows values up to 65,535 practically (higher values may cause issues in older engines).

4.4 Extra Field

This field appears to serve no functional purpose. There is no strong correlation between its value and BLP file usage. Testing shows the value has no visual impact on how textures are processed. A recommended default value is `5` (used by WorldEdit-generated `war3mapMap.blp` minimap files). Other common values include `4`.

4.5 Has Mipmaps

A boolean flag:

- **0**: No mipmaps. Only the full-resolution image at mipmap level 0 is present.
- **Non-zero**: A full complement of mipmaps exists, ending with a 1×1 pixel image at the highest level.

The number of mipmap levels is determined by: `floor(log2(max(width, height))) + 1`, up to a maximum of 16.

Mipmaps are required for textures used with mipmap filtering (e.g., model textures in Warcraft III). The game does **not** auto-generate mipmaps for BLP textures — missing mipmap levels result in transparent black (0, 0, 0, 0) pixels.

5. BLP2 Header

Applies to: BLP2 files (World of Warcraft) **Total header size:** 0x494 (1172) bytes including the palette

```
BLP2Header {
    char[4]    magic           // +0x00  "BLP2" (0x32504C42)
    u32        version         // +0x04  Always 1
    u8         colorEncoding    // +0x08  Encoding type (see table
below)
    u8         alphaBitDepth    // +0x09  Alpha bits: 0, 1, 4, or 8
    u8         alphaType        // +0x0A  Alpha/format type (see
table below)
    u8         hasMipmaps       // +0x0B  0 = none, 1 = generated, 2
= hand-made
    u32        width           // +0x0C  Image width in pixels
    u32        height          // +0x10  Image height in pixels
    u32[16]    mipOffsets       // +0x14  Byte offsets to each mipmap
level
    u32[16]    mipSizes         // +0x54  Byte sizes of each mipmap
level
    u32[256]   palette          // +0x94  256-entry BGRX colour
palette (always present)
}
```

5.1 Color Encoding

Value	Name	Description
0	JPEG	JPEG-compressed content. Palette region stores a shared JPEG header. Rarely used in BLP2; common in BLP1.
1	Palettized	Indexed colour using the 256-entry palette. Each pixel is a 1-byte index. Alpha data follows.
2	DXT	DXT/S3TC block-compressed. Specific format determined by alphaType .
3	BGRA	Uncompressed 32-bit BGRA pixels. Introduced in WoW: Cataclysm. Palette present but unused.
4	BGRA (dup)	Same decompression as 3 ; likely for a different internal pixel format.

5.2 Alpha Type / DXT Format Selection

For DXT-compressed textures ([colorEncoding](#) = [2](#)), the [alphaType](#) field selects the DXT variant:

alphaType	DXT Format	Block Size	Description
0	DXT1 (BC1)	8 bytes	4-bit colour, optional 1-bit alpha. Used when <code>alphaBitDepth</code> is 0 or 1.
1	DXT3 (BC2)	16 bytes	4-bit colour + explicit 4-bit alpha per pixel.
7	DXT5 (BC3)	16 bytes	4-bit colour + interpolated 8-bit alpha.

For palettized textures, `alphaType` typically takes values 2, 4, or 8 but does not affect decoding.

5.3 Mipmap Level and Flags

The `hasMipmaps` field in BLP2 encodes both mipmap presence and generation method:

Value	Name	Description
0	<code>MIPS_NONE</code>	No mipmap levels beyond level 0
1	<code>MIPS_GENERATED</code>	Mipmaps are auto-generated
2	<code>MIPS_HANDMADE</code>	Mipmaps were hand-authored (not commonly supported)

The lower 4 bits encode the mipmap level count/type, and bit `0x10` is an additional flag of unknown purpose.

5.4 Palette

The palette is a 256-entry array of 32-bit values in BGRX byte order:

```
PaletteEntry {
    u8    blue
    u8    green
    u8    red
    u8    padding           // Unused; set to 0 for compressibility
}
```

The palette is **always** present in BLP2 files (1024 bytes at offset 0x94), regardless of whether the encoding uses it. For non-palettized formats, it should be zeroed.

For JPEG-encoded BLP2 files, the palette region is repurposed: the first 4 bytes store the JPEG header size, followed by up to 1020 bytes of JPEG header data.

6. Mipmap Location Table

Applies to: BLP1 and BLP2 (not BLP0)

The mipmap location table consists of two arrays of 16 **u32** values:

```
MipmapLocator {
    u32[16]    mipOffsets      // Byte offset from file start to
each mipmap data chunk
    u32[16]    mipSizes       // Byte size of each mipmap data
chunk
}
```

Size: 128 bytes

Each array index corresponds to a mipmap level: index 0 is the full-size image, index 1 is half resolution, and so on until the 1×1 pixel level.

- If **hasMipmaps** is 0, only index 0 is used.
- If **hasMipmaps** is non-zero, indices 0 through **numMipLevels** - 1 are populated.
- Unused entries should have offset and size of 0.

Layout in BLP1

In BLP1 files, the mipmap locator immediately follows the main header:

Offset	Field
0x1C	mipOffsets[16] (64 bytes)
0x5C	mipSizes[16] (64 bytes)

Layout in BLP2

In BLP2 files, the mipmap locator is embedded within the header structure:

Offset	Field
0x14	mipOffsets[16] (64 bytes)
0x54	mipSizes[16] (64 bytes)

Positioning Rules

There are no strict positioning or ordering requirements for mipmap data chunks within the file. Padding can exist between chunks. However, it is recommended that mipmap data chunks be placed sequentially in ascending mipmap level order with no padding in between.

Mipmap data chunks must be sourced from within the file. Warcraft III does **not** bounds-check the buffers used when reading past EOF, resulting in undefined and crash-prone behaviour. If mipmap data extends beyond EOF, the read should be truncated to EOF and a warning generated.

BLP0 External Mipmaps

BLP0 files have **no** mipmap location table. Instead, mipmap data is stored in separate companion files:

- The full-scale image (level 0) is in `<basename>.b00`
- Each subsequent level uses the extension `.bXX` where `XX` is the zero-padded mipmap level number (e.g., `.b08` for level 8, `.b10` for level 10)
- The mipmap data chunk size equals the file size of the companion file

With the exception of how mipmap data is sourced, BLP0 and BLP1 are identical. A lossless BLP0-to-BLP1 converter can provide BLP0 support for a BLP1-only reader.

7. Content Headers

Content headers follow the mipmap location table (in BLP1) and precede the mipmap data. The content header type is determined by the `content` field (BLP0/BLP1) or `colorEncoding` field (BLP2).

7.1 JPEG Content Header

Applies to: `CONTENT_JPEG` (BLP0/BLP1) or `colorEncoding = 0` (BLP2)

```
JPEGContentHeader {
    u32      jpegHeaderSize      // Size in bytes of the shared JPEG
header chunk
    u8[]      jpegHeaderChunk    // Shared JPEG header data
(jpegHeaderSize bytes)
}
```

The `jpegHeaderChunk` is prepended to each mipmap data chunk to form a complete JFIF image for decoding.

Size limits: The maximum valid `jpegHeaderSize` is **624 bytes** (0x270). Larger values are prone to causing image corruption and crashes in Warcraft III 1.27b due to a fixed-size buffer. When writing BLP files with JPEG content:

- Values larger than 624 bytes should trigger a warning.
- If `jpegHeaderSize` causes the chunk to extend past EOF, truncate to available data and warn.
- The header chunk can be empty (size 0) if mipmaps contain complete JPEG files.

In BLP2, the JPEG header data occupies the palette region at offset 0x94: the first 4 bytes are the size, followed by up to 1020 bytes of header data.

7.2 Direct Content Header (Palette)

Applies to: `CONTENT_DIRECT` (BLP0/BLP1) or `colorEncoding = 1` (BLP2)

```
DirectContentHeader {
    u32[256]  palette            // 256-entry colour look-up table
(1024 bytes)
}
```

Each palette entry is a 32-bit value containing BGR colour components:

Byte	Component
0	Blue
1	Green
2	Red
3	Padding (unused, typically 0)

The padding byte does not alter the decoded image. It can be set to values that improve file compressibility.

The palette must **always** be present for **CONTENT_DIRECT** / palettized files, as well as for BLP2 DXT and BGRA files (where it is present but unused).

7.3 Content Header Position

Version	JPEG Content	Direct Content
BLP0	Offset 0x1C (after header, no mipmap locator)	Offset 0x1C
BLP1	Offset 0x9C (after mipmap locator)	Offset 0x9C
BLP2	Palette region at 0x94 (see §7.1)	Palette at 0x94 (part of header)

8. Mipmap Data

Mipmap data chunks contain the encoded pixel data for each mipmap level. The encoding and decoding process depends on the content type.

8.1 Mipmap Dimensions

Each mipmap level halves both dimensions of the previous level. Dimensions are computed as:

```
mipmapWidth(level)  = max(1, width  >> level)
mipmapHeight(level) = max(1, height >> level)
mipmapPixelCount(level) = mipmapWidth(level) * mipmapHeight(level)
```

The total number of mipmap levels (when mipmaps are present) is:

```
numMipLevels = floor(log2(max(width, height))) + 1
```

This is capped at 16 by the locator table size.

8.2 Pixel Ordering

In all encoding modes, pixels are ordered in **scan lines from left to right, top to bottom**. The scan line length and count are derived from the width and height after applying mipmap scaling.

9. Encoding Formats

9.1 JPEG Content

Content type: `CONTENT_JPEG` (0) for BLP0/BLP1, `colorEncoding = 0` for BLP2

Each mipmap data chunk has the format:

```
JPEGMipmapData {  
    u8[]          jpegChunk          // mipSizes[level] bytes  
}
```

To decode a mipmap level:

1. Concatenate `jpegHeaderChunk` + `jpegChunk` to form a complete JFIF image.
2. Decode the resulting JPEG data using a compliant JPEG codec.
3. Interpret the decoded components as **BGRA** (not the standard Y'CbCr).

Critical details:

- The JPEG file contains **4-channel BGRA** colour components. This is **non-standard**; most JPEG codecs expect Y'CbCr and will automatically perform colour conversion. The raw component values must be used directly **before** any codec-applied conversion.
- Chroma subsampling is not possible (or will produce bad results) since BGRA components are used rather than Y'CbCr.
- Blizzard used the (now discontinued) Intel Imaging Framework to produce/decode JPEG content, meaning official BLP files contain its signature in the header chunk.
- The JPEG file defines its own internal width and height, which should match the expected mipmap dimensions. Some tools produce mismatched dimensions for high mipmap levels. The image should be cropped or padded (with transparent black) to match the expected size.

Alpha handling for JPEG:

- When `alphaBitDepth` is 0, the alpha channel should be ignored (fully opaque). However, Warcraft III has inconsistent behaviour: UI images correctly treat 0-bit alpha as opaque, but model textures still blend using the raw alpha values.
- **Recommendation:** When writing `CONTENT_JPEG` with `alphaBitDepth = 0`, always set the alpha band to `0xFF` (fully opaque) to avoid transparency artefacts on model textures.

9.2 Palettized / Indexed Content (RAW1)

Content type: `CONTENT_DIRECT` (1) for BLP0/BLP1, `colorEncoding = 1` for BLP2

Each mipmap data chunk has the format:

```
IndexedMipmapData {
    u8[]      indexedRGB      // mipmapPixelCount bytes (palette
indices)
    u8[]      indexedAlpha    // ceil(mipmapPixelCount *
alphaBitDepth / 8) bytes
}
```

Decoding RGB: Each byte in `indexedRGB` is an index into the 256-entry palette. Use the index to retrieve the BGR colour components from the palette.

Decoding Alpha: The `indexedAlpha` array contains packed alpha values at the precision specified by `alphaBitDepth`:

alphaBitDepth	Packing	Description
0	(no data)	No alpha channel; all pixels fully opaque (alpha = 255)
1	8 pixels per byte	Each bit is 0 (transparent) or 1 (opaque), scaled to 0 or 255 in 8-bit. Ordered LSB-first within each byte
4	2 pixels per byte	Each nibble is a 4-bit alpha value (0–15), scaled to 8-bit. Low nibble first
8	1 pixel per byte	Direct 8-bit alpha value (0–255)

Alpha bit packing examples (MSB ← → LSB, numbers indicate sequential pixel index):

```
8-bit:  [11111111]           → 1 pixel per byte
4-bit:  [2222 1111]           → 2 pixels per byte (low
nibble = first pixel)
1-bit:  [8 7 6 5 4 3 2 1]     → 8 pixels per byte (bit 0 =
first pixel)
```

Scaling alpha to 8-bit:

- 1-bit: `alpha8 = bit ? 255 : 0`
- 4-bit: `alpha8 = (nibble << 4) | nibble` (or equivalently `nibble * 17`)

Size validation: The total size of both arrays should match `mipSizes[level]`. If the expected chunk size does not match, resize the buffer appropriately and generate a warning.

Note: In Warcraft III, the number of bytes read for indexed mipmap data does **not** depend on `mipSizes[level]` — it always reads the number of bytes required based on the pixel count. The mipmap locator size may be inconsistent in some files.

9.3 DXT / S3TC Content

Content type: `colorEncoding = 2` (BLP2 only)

DXT-compressed mipmap data consists of 4×4 pixel blocks, ordered left-to-right, top-to-bottom. The specific DXT variant is determined by `alphaType`.

Each mipmap level's data size can be calculated:

```
blocksWide = (mipmapWidth + 3) / 4
blocksHigh = (mipmapHeight + 3) / 4
totalBlocks = blocksWide * blocksHigh

DXT1:      dataSize = totalBlocks * 8
DXT3/DXT5: dataSize = totalBlocks * 16
```

Note: Some existing BLP files have incorrect `mipSizes` values for lower mipmap levels. Always use the calculated size to allocate the decode buffer. Read `mipSizes[level]` bytes from the file into the buffer and zero-fill any remaining bytes.

9.3.1 DXT1 (BC1) — `alphaType = 0`

Block size: 8 bytes (64 bits)

```
DXT1Block {
    u16    color0           // RGB 5:6:5 endpoint colour 0
    u16    color1           // RGB 5:6:5 endpoint colour 1
    u32    lookupTable      // 16 × 2-bit indices (4×4 pixel block)
}
```

Colour palette derivation:

The two 16-bit values are decoded as RGB 5:6:5 colours (bits: RRRRRGGGGGGBBBBB, least significant = blue) to form the first two palette entries.

- If `color0 > color1`: Four-colour mode. The other two palette colours are interpolated:
 - `color2 = (2 * color0 + color1) / 3`
 - `color3 = (color0 + 2 * color1) / 3`
- If `color0 <= color1`: Three-colour + transparent mode. The fourth entry is reserved:
 - `color2 = (color0 + color1) / 2`
 - `color3 = transparent black` (if `alphaBitDepth >= 1`) or `opaque black` (if `alphaBitDepth = 0`)

The 32-bit lookup table encodes 16 two-bit indices (for the 4×4 block), specifying which palette colour each pixel uses. Indices are packed LSB-first, scanning left-to-right, top-to-bottom within the

block.

9.3.2 DXT3 (BC2) — `alphaType = 1`

Block size: 16 bytes (128 bits)

```
DXT3Block {
    u64    alphaData          // 16 × 4-bit explicit alpha values
    u16    color0             // RGB 5:6:5 endpoint colour 0
    u16    color1             // RGB 5:6:5 endpoint colour 1
    u32    lookupTable        // 16 × 2-bit colour indices
}
```

The alpha portion contains 16 four-bit alpha values (one per pixel in the 4×4 block), packed LSB-first. Each 4-bit value represents alpha in the range 0–15, scaled to 0–255.

The colour portion is identical to DXT1 in four-colour mode (no reserved/transparent entry).

9.3.3 DXT5 (BC3) — `alphaType = 7`

Block size: 16 bytes (128 bits)

```
DXT5Block {
    u8      alpha0            // Alpha endpoint 0
    u8      alpha1            // Alpha endpoint 1
    u8[6]   alphaIndices      // 16 × 3-bit alpha lookup indices (48
bits)
    u16     color0            // RGB 5:6:5 endpoint colour 0
    u16     color1            // RGB 5:6:5 endpoint colour 1
    u32     lookupTable       // 16 × 2-bit colour indices
}
```

Alpha lookup table derivation:

The two 8-bit alpha endpoint values form the first two entries. Remaining entries interpolated:

- If `alpha0 > alpha1`: Eight-entry table:
 - `alpha2-alpha7` = linearly interpolated between `alpha0` and `alpha1`
- If `alpha0 <= alpha1`: Six-entry table, plus two reserved:
 - `alpha2-alpha5` = linearly interpolated
 - `alpha6 = 0` (fully transparent)
 - `alpha7 = 255` (fully opaque)

The 48-bit index block contains 16 three-bit lookup indices, packed LSB-first, specifying which table entry each pixel uses.

The colour portion is identical to DXT1 in four-colour mode.

9.4 Uncompressed BGRA Content (RAW3)

Content type: `colorEncoding = 3` (or `4`) in BLP2 only

Each mipmap data chunk stores raw 32-bit pixels:

```
BGRAMipmapData {  
    BGRAPixel[mipmapPixelCount]    // 4 bytes per pixel  
}
```

```
BGRAPixel {  
    u8    blue  
    u8    green  
    u8    red  
    u8    alpha  
}
```

Pixels can be read directly and loaded into a texture without further decoding. The colour palette is present in the file header but unused. The `alphaBitDepth` field may take on unusual values (e.g., 136) in this mode and appears to serve as bit flags rather than a true depth value.

10. Version Differences

10.1 BLP0 vs BLP1

Feature	BLP0	BLP1
Magic	"BLP0"	"BLP1"
Mipmap locator	Absent	Present (128 bytes at 0x1C)
Mipmap storage	External files (.b00–.b15)	Internal (within same file)
Content header offset	0x1C	0x9C
Supported content types	JPEG, Palettized	JPEG, Palettized
Maximum dimension	512 (effective)	65,535 (limited to 512 by WC3 before 1.27b)

All other structures and encoding logic are identical between BLP0 and BLP1.

10.2 BLP1 vs BLP2

Feature	BLP1	BLP2
Magic	"BLP1"	"BLP2"
Header flags layout	u32 fields (content, alphaBitDepth, extra, hasMipmaps)	u8 fields (colorEncoding, alphaBitDepth, alphaType, hasMipmaps)
Mipmap locator position	Separate (0x1C)	Inline in header (0x14)
Palette position	After mipmap locator (0x9C)	After mip sizes at fixed offset (0x94)
extra field	Present	Absent
Supported encodings	JPEG, Palettized	JPEG, Palettized, DXT1, DXT3, DXT5, BGRA
Version field	Implicit from magic	Explicit (always 1)
DXT/S3TC support	No	Yes (via colorEncoding = 2)
Uncompressed BGRA	No	Yes (via colorEncoding = 3 or 4)

10.3 Summary of Header Offsets

Field	BLP0	BLP1	BLP2
Magic	0x00	0x00	0x00
Content / ColorEncoding	0x04 (u32)	0x04 (u32)	0x08 (u8)
AlphaBitDepth	0x08 (u32)	0x08 (u32)	0x09 (u8)
AlphaType	—	—	0x0A (u8)
HasMipmaps	0x18 (u32)	0x18 (u32)	0x0B (u8)
Width	0x0C	0x0C	0x0C
Height	0x10	0x10	0x10
Extra	0x14	0x14	—
Version	—	—	0x04 (u32)
MipOffsets	—	0x1C	0x14
MipSizes	—	0x5C	0x54
Palette	0x1C*	0x9C*	0x94

* For **CONTENT_DIRECT** only. For **CONTENT_JPEG**, the JPEG content header occupies this position instead.

11. Implementation Notes

11.1 Parsing Strategy

```

data = readFile(path)
magic = readU32(data, 0)

switch (magic) {
    case BLP0_MAGIC: version = 0; break;
    case BLP1_MAGIC: version = 1; break;
    case BLP2_MAGIC: version = 2; break;
    default: error("Not a BLP file");
}

if (version < 2) {
    // BLP0/BLP1 path
    content      = readU32(data, 0x04)
    alphaBits    = readU32(data, 0x08)
    width        = readU32(data, 0x0C)
    height       = readU32(data, 0x10)
    extra        = readU32(data, 0x14)
    hasMipmaps   = readU32(data, 0x18)

    if (version >= 1) {
        mipOffsets = readU32Array(data, 0x1C, 16)
        mipSizes   = readU32Array(data, 0x5C, 16)
    }

    headerEnd = (version >= 1) ? 0x9C : 0x1C

    if (content == CONTENT_DIRECT) {
        palette = readPalette(data, headerEnd) // 256 x u32
    } else {
        jpegHeaderSize = readU32(data, headerEnd)
        jpegHeader      = readBytes(data, headerEnd + 4, jpegHeaderSize)
    }
} else {
    // BLP2 path
    blp2Version    = readU32(data, 0x04) // must be 1
    colorEncoding  = readU8(data, 0x08)
    alphaBits      = readU8(data, 0x09)
    alphaType      = readU8(data, 0x0A)
    hasMipmaps     = readU8(data, 0x0B)
    width          = readU32(data, 0x0C)
    height         = readU32(data, 0x10)
    mipOffsets     = readU32Array(data, 0x14, 16)
    mipSizes       = readU32Array(data, 0x54, 16)
    palette        = readU32Array(data, 0x94, 256)
}

```

11.2 Mipmap Generation

When producing mipmaps, an algorithm appropriate for the source colour space should be used:

- Simple averaging is only appropriate for linear RGB colour space.
- For sRGB images (the practical convention for BLP), convert to linear, average, then convert back — or use a perceptually correct downsampling algorithm.
- For higher mipmap levels (very small images), visual correctness can be sacrificed for file size. For indexed content, higher mipmap data chunks can point into lower-level mipmap data within the file.

11.3 Warcraft III Texture Size Limits

Version	BLP Limit	Notes
WC3 before 1.27b	512×512	Hard-coded limit; larger BLP files rejected
WC3 1.27b–1.28	512×512 (effective)	Larger files load but only the ≤512×512 mipmap is used
WC3 1.29+	65,535×65,535	Engine limit removed; GPU capability dependent

TGA files loaded by Warcraft III are not subject to these BLP-specific limits with mipmaps auto-generated by the driver.

11.4 JPEG Content Quirks

- Blizzard's encoder produces JPEG files with the Intel Imaging Framework signature in the shared header. The presence of this signature is optional.
- Some official Warcraft III BLP files are malformed: they declare 0-bit alpha but contain non-opaque alpha values in certain mipmap levels. Always write `0xFF` alpha when `alphaBitDepth = 0`.
- If any mipmap level of a JPEG-content BLP fails to decode, the entire texture is considered invalid in Warcraft III. This is a common failure mode for older third-party BLP conversion tools.

11.5 Palette Entry Details

The palette stores 256 entries of 4 bytes each. While many sources describe these as BGRA, the fourth byte is **padding** and does not serve as an alpha channel. Alpha data for palettized images always comes from the separate `indexedAlpha` array within each mipmap data chunk. Setting the padding byte to 0 is recommended.

11.6 Robustness Recommendations

Scenario	Recommendation
Invalid <code>content</code> / <code>colorEncoding</code> value	Default to JPEG; generate warning
Invalid <code>alphaBitDepth</code> for encoding type	Treat as 0 (opaque); generate warning
<code>jpegHeaderSize</code> > 624	Load normally but warn; may crash WC3 1.27b
<code>jpegHeaderSize</code> extends past EOF	Truncate JPEG header to available data; warn
Mipmap data extends past EOF	Truncate to available data; warn
DXT mipmap <code>mipSizes</code> doesn't match calculated	Use calculated size for buffer, read <code>mipSizes</code> bytes; zero-fill remainder
JPEG mipmap fails to decode	Entire texture invalid; do not load
Mismatched JPEG internal dimensions	Crop/pad to expected mipmap dimensions
BLP0 companion file missing	Skip that mipmap level; warn

Appendix A — Mipmap Scaling Logic

Given the full-scale image dimensions `width` × `height`, each mipmap level `L` has:

```
w(L) = max(1, floor(width / 2^L))
h(L) = max(1, floor(height / 2^L))
```

Equivalently:

```
w(L) = max(1, width >> L)
h(L) = max(1, height >> L)
```

The total number of valid mipmap levels is:

```
numLevels = floor(log2(max(width, height))) + 1
numLevels = min(numLevels, 16)           // Capped by locator
table
```

Example: For a 256×128 image:

Level	Width	Height	Pixels
0	256	128	32,768
1	128	64	8,192
2	64	32	2,048
3	32	16	512
4	16	8	128
5	8	4	32
6	4	2	8
7	2	1	2
8	1	1	1

Total: 9 mipmap levels.

Appendix B — DXT Block Compression Details

Block Layout in Mipmap Data

DXT-compressed mipmaps consist of 4×4 pixel blocks arranged in scan-line order:

```
blocksWide = ceil(width / 4) = (width + 3) / 4
blocksHigh = ceil(height / 4) = (height + 3) / 4
```

Blocks are stored left-to-right, top-to-bottom. Pixels within each block also follow left-to-right, top-to-bottom ordering.

For images whose dimensions are not multiples of 4, the rightmost and/or bottom blocks contain padding pixels that extend beyond the image boundary. These extra pixels are encoded normally but should be discarded on decode.

Minimum Block Count

Even a 1×1 mipmap requires at least one 4×4 block:

```
DXT1:  minimum 8 bytes
DXT3:  minimum 16 bytes
DXT5:  minimum 16 bytes
```

RGB 5:6:5 Colour Decoding

The 16-bit colour values in DXT blocks encode RGB in 5:6:5 bit layout:

```
r5 = (color >> 11) & 0x1F      // 5 bits
g6 = (color >>  5) & 0x3F      // 6 bits
b5 = (color >>  0) & 0x1F      // 5 bits

// Expand to 8-bit:
r8 = (r5 << 3) | (r5 >> 2)
g8 = (g6 << 2) | (g6 >> 4)
b8 = (b5 << 3) | (b5 >> 2)
```

DXT5 Alpha Index Unpacking

The 48-bit (6-byte) alpha index block contains 16 three-bit indices packed LSB-first:

```
// bytes[0..5] contain 48 bits
for (i = 0; i < 16; i++) {
    bitOffset = i * 3
    byteIndex = bitOffset / 8
    bitShift  = bitOffset % 8

    index = (bytes[byteIndex] >> bitShift) & 0x07
    if (bitShift > 5)
        index |= (bytes[byteIndex + 1] << (8 - bitShift)) & 0x07
}
```

Credits & References

Primary Sources:

- [wowdev.wiki — BLP](#): Community-maintained BLP2 documentation for World of Warcraft reverse engineering
- [Dr Super Good — BLP Specifications \(WC3\)](#): Comprehensive Warcraft III BLP0/BLP1 specification based on extensive reverse engineering and hex-level analysis
- [Wowpedia — BLP Files](#): BLP2 format overview with encoding details and sample files
- [zloy-tulen/image-blp](#): Rust crate implementing BLP0/BLP1/BLP2 encoding and decoding for all known formats

Additional References:

- [Wikipedia — S3 Texture Compression](#): DXT/BC block compression format details
- Microsoft DirectX documentation for DXT1 (BC1), DXT3 (BC2), and DXT5 (BC3) format specifications